



# Operating System

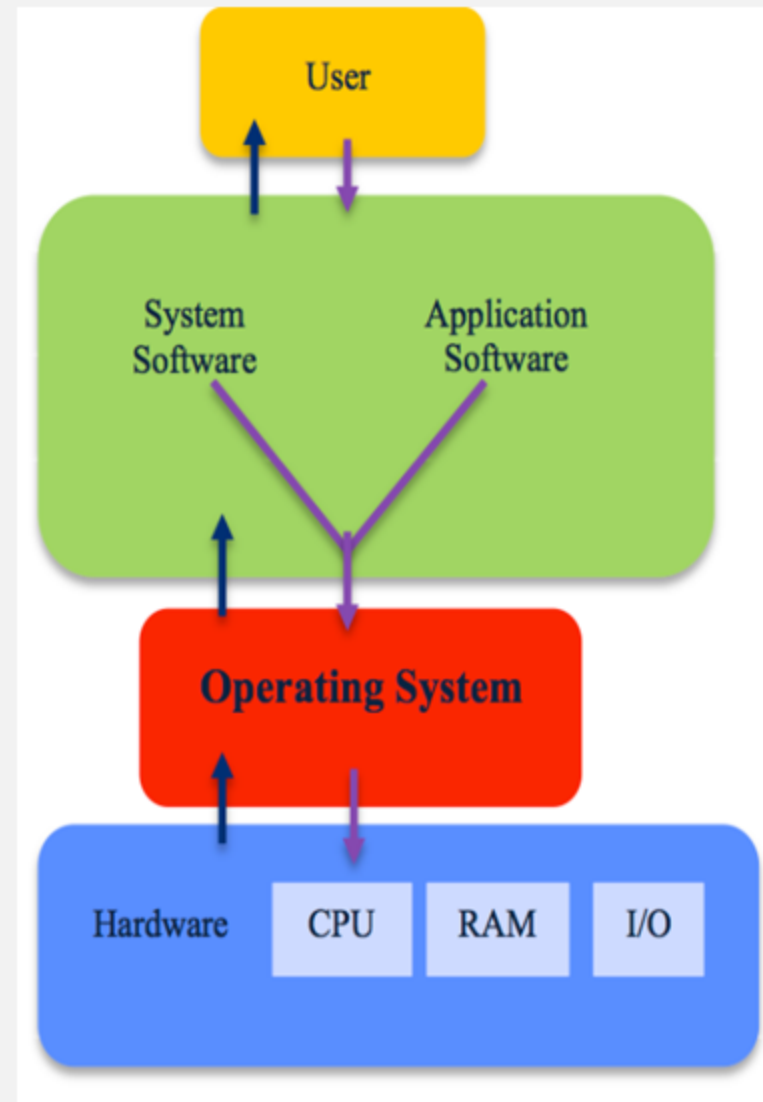
- ▶ Definition of OS
- ▶ Functionality
- ▶ Types of OS
- ▶ Examples of OS
- ▶ Some concepts in OS
  - ▶ Single programming
  - ▶ Multi programming
- ▶ Memory
- ▶ Kernel

## Operating System works as :

- ❖ Interface between user and hardware.
- ❖ Resource allocator.
- ❖ Program controller.

## System S/w

- ❖ System software is a type of computer program that is designed to run computer's hardware and application programs.
- ❖ It is an interface between the hardware and user applications.



- **The BIOS (basic input/output system) ::**

This system software ,gets the computer system started after it's turned on and manages the data flow between the OS and attached devices, such as the hard drive, video adapter, keyboard,mouse and printer.

- **The boot program ::**

This system software loads the OS into the computer's main memory or random access memory (RAM).

- **An assembler ::**

Takes basic computer instructions and converts them into a pattern of bits that the processor of computer can use to perform its basic operations.

- **A device driver ::**

Controls a particular type of device that is attached to your computer, such as a keyboard or mouse.

Additionally, system software can also include **system utilities** like---

- ✓ Disk Defragmenter
- ✓ Compilers
- ✓ Debuggers.

# Popular Operating Systems



# Functions Of OS

**Security** : OS uses password protection to protect user data and similar other techniques to prevent unauthorized access to programs and user data.

**Resource Managment** : OS ensures the proper use of all the available resources .

**Process Managment** : In a multi programming environment, the OS decides the order in which processes will have access to the processor, and how much processing time each process has. OS keeps track of the status of processes



**Job accounting** : Operating system Keeps track of time and resources used by various tasks and

**Error detection** : Operating system constantly monitors the system to detect errors and avoid malfunctioning of computer system.

**Memory Managment** : The operating system manages the Primary Memory or Main Memory.

**Coordination between S/Ws and users** : Operating systems also coordinates and assigns interpreters, compilers, assemblers and other software to the various users of the computer systems.

**Command Interpreter**: Major functions of OS is to interpret the commands. OS provides an **interface** to the users for the communication with the Computer. Command interpreter is also known as a **Shell**. Command interpreter provides a user interface to hide hardware details of the system from its users, making system more user friendly.

The two commonly used methods, provided by operating systems, for interfacing are

1. Command line Interface (**CLI**)
2. Graphical User Interface (**GUI**).



- Windows (GUI based, PC)
- GNU/Linux (Personal, Workstations)
- macOS (Macintosh), used for Apple's personal computers and work stations (MacBook, iMac).
- Android (Google's Operating System for smartphones/tablets/smartwatches)
- iOS (Apple's OS for iPhone, iPad and iPod Touch)

Popular OSs for computers are::

- ✓ Windows 10
- ✓ Mac OS X
- ✓ Ubuntu

Popular network/server OSs are::

- ✓ Ubuntu Server
- ✓ Windows Server
- ✓ Red Hat Enterprise

Popular mobile OSs are::

- ✓ iPhone OS
- ✓ Android OS
- ✓ Windows Phone OS

# Some concepts in OS

## Single Programming OS:

- ❖ Only one application program runs on top of OS.
- ❖ Single user operating system is that in which one user works on one interface.
- ❖ In these systems only one user is active at a time.

These systems use less resources and they have less complexity so it is easy to maintain and debug these systems.

- ❖ The system has a less chance to get damage , since number of requests from hardware and software is few.

This is Single-User , Single-Tasking OS.

-- e.g MS-DOS

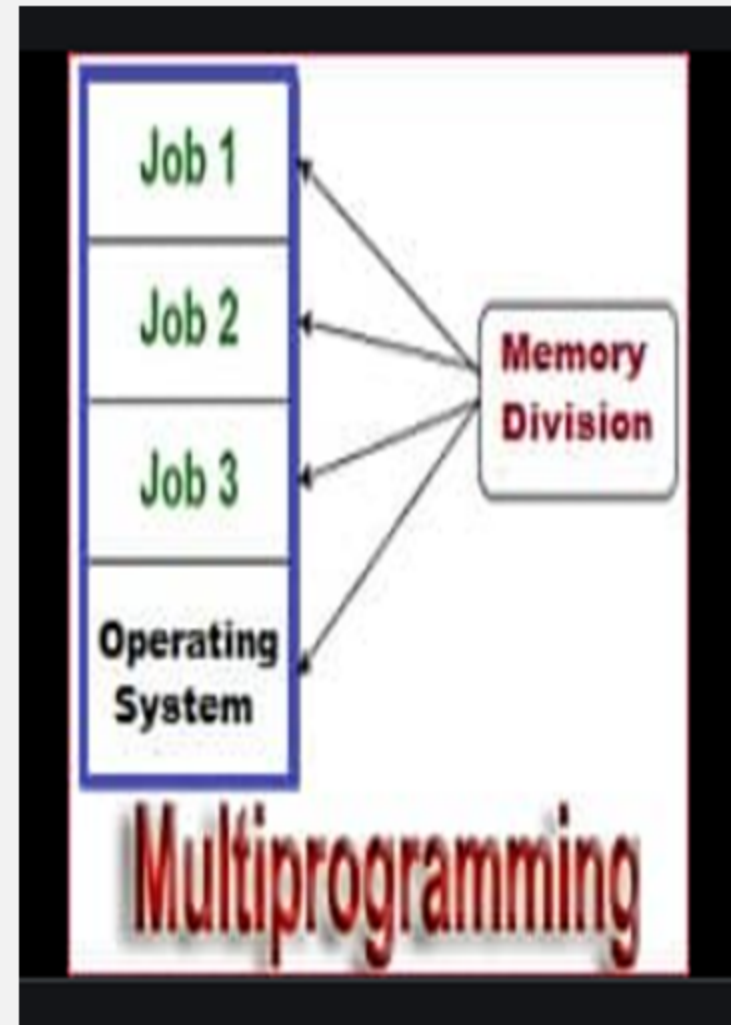


## Disadvantage of Single Programming System:

- Major disadvantage of this system is that, it is very slow and it fails to handle work load efficiently.
- This OS has lot of limits and also CPU utilization is very poor , since CPU has to wait for another program to be brought into the memory after completion of execution of first program.
- Hence the concept of Multiprogramming evolved in Operating System.

# Concepts in OS

- ❖ In Multiprogramming, more than one program gets loaded in memory along with OS.
- ❖ CPU will remain busy always.
- ❖ Processor switches rapidly between the programs available in the memory.
- ❖ In this case , OS starts executing next job when--
  1. Program terminates
  2. Program is waiting for i/o services.
- ❖ This is non-preemptive system. CPU will process till the job is completed. CPU will switch to other job in case if the job goes for i/o functions or job terminates logically.
- ❖ System actually processes only one program at a time. The switching between programs is very fast .



-e.g Batch Processing system.

# Multiprocessing/Time Sharing OS/Multitasking

- ❖ Idea is to process multiple program simultaniously.
- ❖ Time slice/quantum is defined by OS.
- ❖ In this case OS will switch to other job when-----
- ✓ Time slice/quantum ends.
- ✓ Program is waiting for i/o operations to complete.
- ✓ Program terminates.

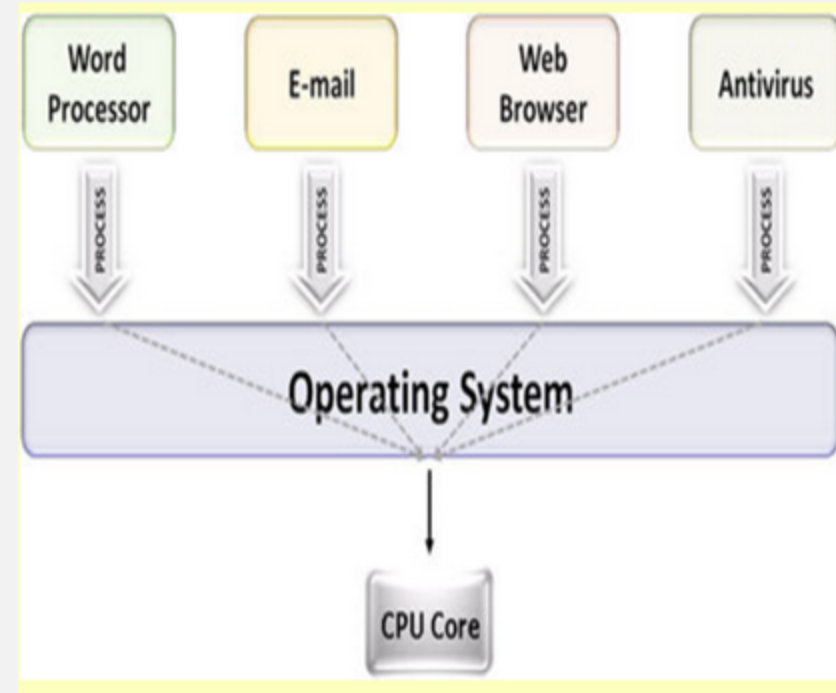
--Multiple programs are brought into the memory.

--In ideal case, CPU will process each program for definite time quantum and then switch to the other program.

--This switching is very fast and though only one program is executed at any point of time, the user gets the effect, that multiple programs are running in parallel.

--The CPU is never idle and is utilized to its optimum capacity.

-- This type of a system requires extra h/w called as a Timer.



This increases speed and throughput of the system.



e.g

- RAM(Random Access Memory)
- RAM is also called as main/base memory.

## Temporary Memory

- ❖ This type of memory , as the name says is temporary.
- ❖ The data in this memory will be cleared if system shuts down or there is a power-cut.
- ❖ Since the memory is erased due to power failure , it is called as Volatile memory.
- ❖ All the processing and loading of OS is done in this memory.
- ❖ CPU can directly access this memory.

## Permanent Memory

- ❖ Memory is permanent.
- ❖ Data stored stays permanent and needs to be deleted explicitly .
- ❖ If the data is to be processed from this memory, it is brought into RAM for processing.

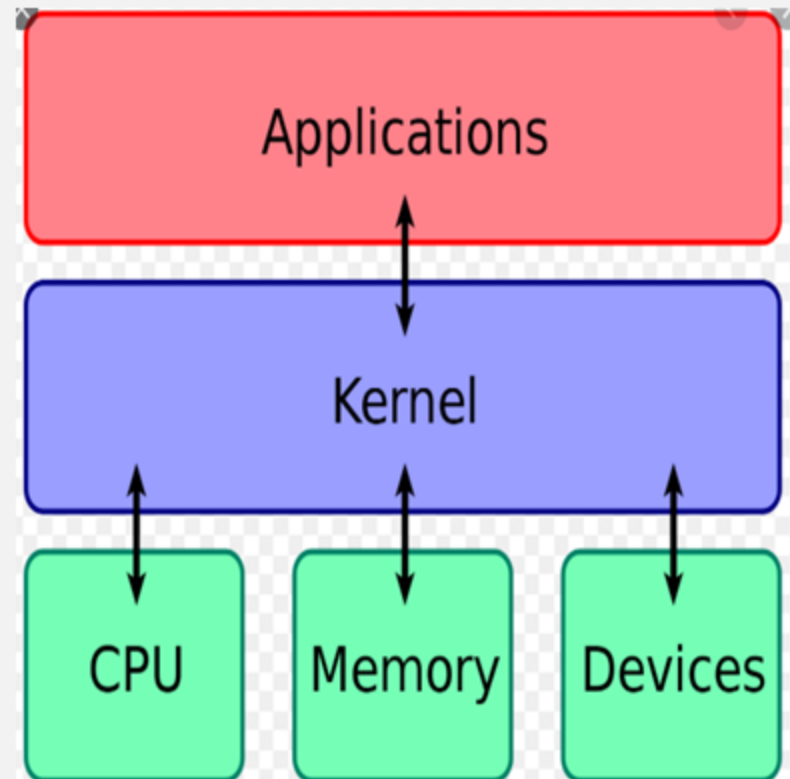


eg.

- ROM (Read Only Memory).
- All important programs required to start the computer system are stored in ROM.



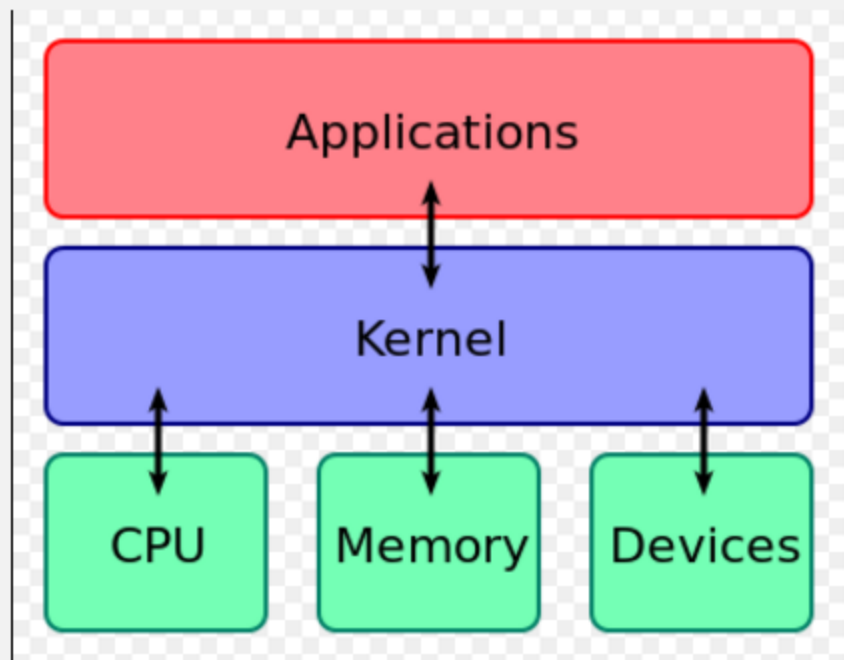
- ❖ Kernel is said to be the "Heart of OS".
- ❖ A kernel is the core component of an operating system which performs all major tasks which we classify as the functions of Operating System  
The main tasks of the kernel are :
  - Process management
  - Device management
  - Memory management
  - Interrupt handling
  - I/O communication
  - File system...etc
- ❖ It is the software responsible for running programs and providing secure access to the machine's hardware.
- ❖ The sole aim of the kernel is to **manage the communication between the software** (user level applications) **and the hardware** (CPU, disk memory etc).



- ❖ **System calls** are handled by Kernel.
- ❖ It is the first program to get loaded in memory and runs till system shuts down. It talks to h/w of the system and most important, it "Handles interrupts".

- When a *process* makes requests of the kernel, the request is called a *system call*.
- Various kernel designs differ in how they manage system calls and *resources*.

## Simple Kernel Structure

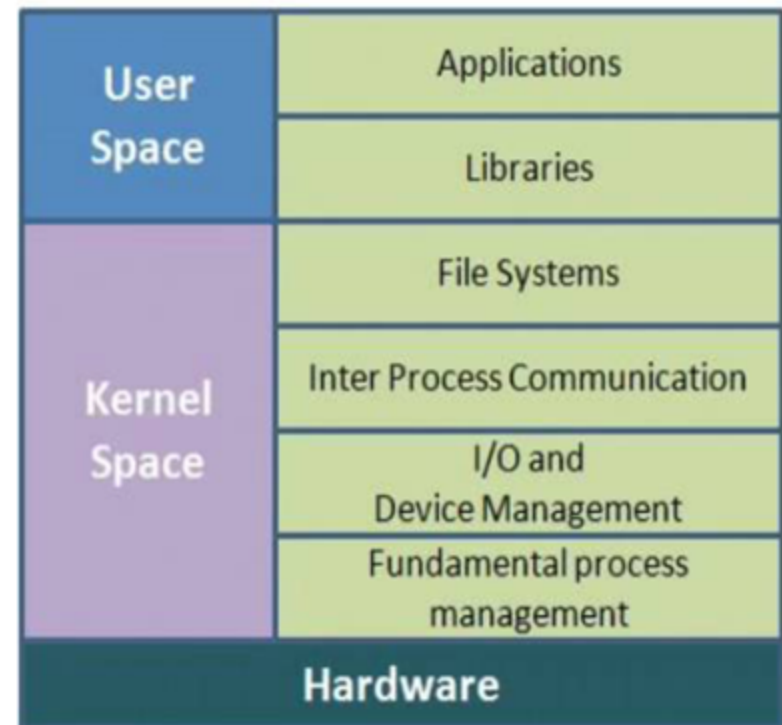


- In this structure there is no distinguish between kernel mode & user mode.
- All the programs can directly interact with OS & H/W.
- Eg. DOS

# Types of Kernel

- ❖ In this structure all the OS tasks(instructions) will run as one process in kernel mode.
- ❖ Application program will run in user mode & can access services through system call interface.
- ❖ When entire / most of the OS programs reside in kernel space it is called as Monolithic Kernel.
- ❖ This Kernel is said to be a Bulkey kernel, as all the programs reside in the same memory space as that of kernel.
- ❖ But lesser switching between the program space is required since all programs share same memory space.
- ❖ Also User services and kernel services

## Monolithic Kernel



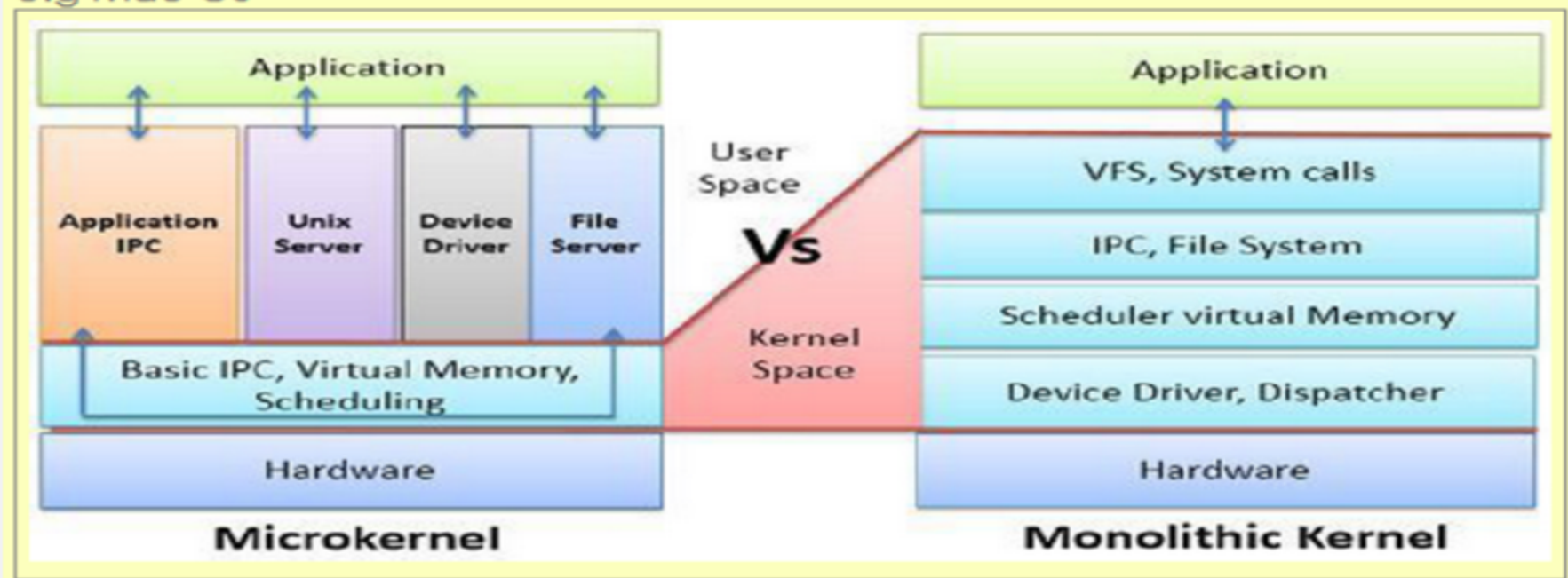
--Drawback: It will increase the size & complexity of kernel.

eg.

-Unix Operating System

- ❖ Instead of having all OS task running in kernel mode, only primitive task of OS will run in kernel mode.
- ❖ All other program/services of OS(networking, process mgmt, etc....) will run in user space.

--Eg. MAC

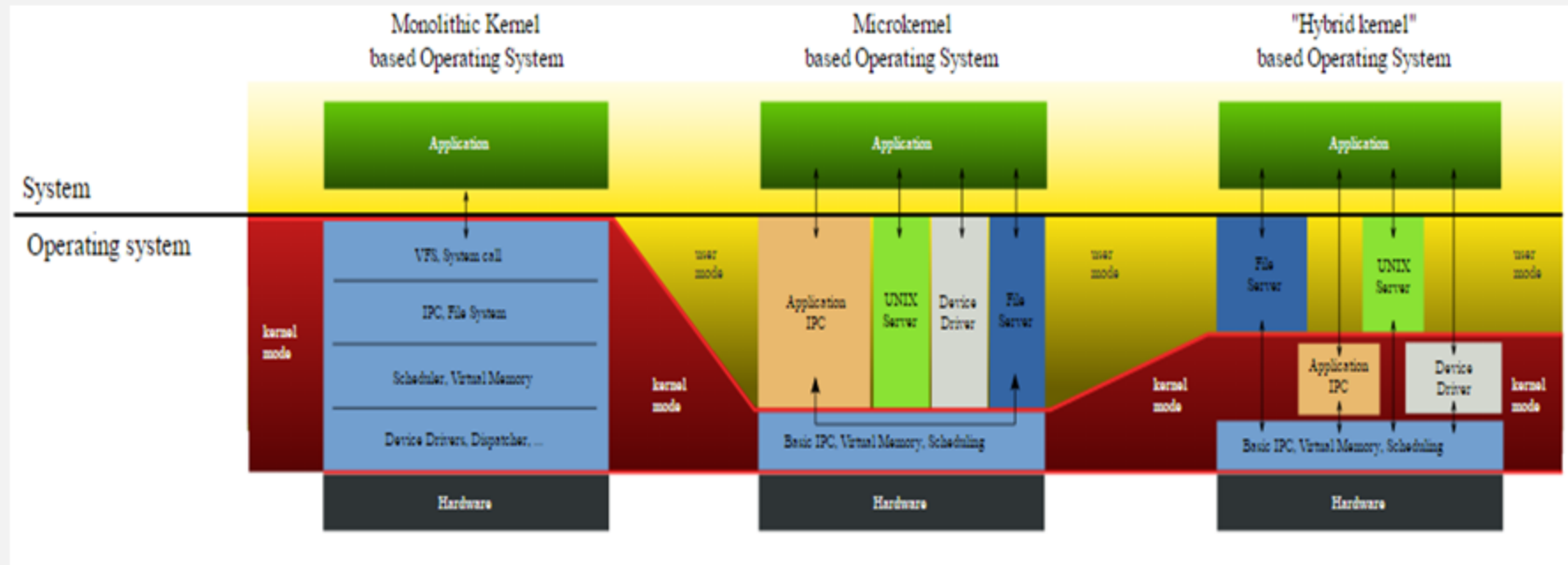
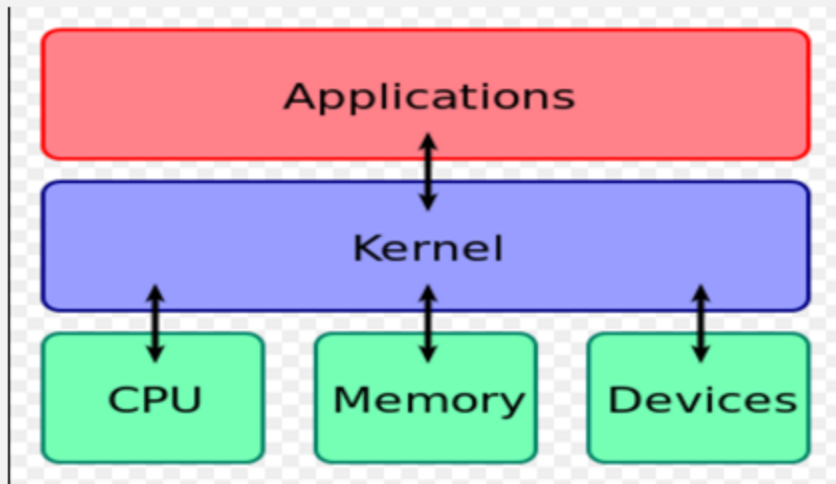


- ❖ Microkernel is the one in which user services and kernel services are kept in separate address space.
- ❖ In this type of Kernel, more switching is required.
- ❖ But the Kernel is less bulky .

- *Exokernel* provide minimal abstractions, allowing low-level hardware access.
- Kernel is limited to extending resources to sub operating systems called *LibOS's*. Resulting in a very small, fast kernel environment.
- In this OS tasks are designed in the form of *modules*, when ever required in future, those modules will be loaded in memory.

- (*modified microkernels*) are much like pure microkernels, except that they include some additional code in kernelspace to increase performance.





## Operating System Structure::

- |   |                        |                              |
|---|------------------------|------------------------------|
| ❖ | Simple Structure       | DOS                          |
| ❖ | Monolithic             | UNIX                         |
| ❖ | Microkernel            | Mac                          |
| ❖ | Object oriented kernel | Linux, Solaris Unix, Windows |

| Monolithic kernel                                                                                                                       | Microkernel                                                                                                                   |
|-----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| All the parts of a kernel like the Scheduler, File System, Memory Management, Networking Stacks, Device Drivers, etc are in the kernel. | Only the very important parts like IPC, basic scheduler, basic memory handling, basic I/O primitives etc., are in the kernel. |
| Monolithic kernels generally are bulky.                                                                                                 | The size the of the microkernel is small.                                                                                     |
| Adding a new feature to a monolithic system means recompiling the whole kernel.                                                         | Adding a new feature to a microkernel doesn't require recompiling the whole kernel.                                           |
| The device driver resides in the kernel space, it makes monolithic kernel less secure.                                                  | The microkernel is more secure as device driver resides in user-space.                                                        |
| Slow execution, as context switch is involved from userspace to kernel space.                                                           | Fast execution, as context switch is not involved from userspace to kernel space.                                             |
| Examples - Linux, FreeBSD, OpenBSD, NetBSD, Solaris, OS-9, AIX, HP-UX, DOS, Microsoft Windows (95,98,Me), OpenVMS, XTS-400 etc.         | Examples - QNX, Integrity, Pik, Symbian, L4Linux, Singularity, K42, Mac OS X, HURD, Minix,                                    |

- ❖ Signal generated by hardware or software when a process or an event needs immediate attention is called as Interrupt.
- ❖ It alerts the processor to a high priority process requiring interruption of the current working process.

```
#include<stdio.h>
int main()
{
    printf("Hello World");           /* i/o call. interrupt generated
}
-----Handled by kernel-----
interrupt called
    write()                         /*accesses o/p device
end                                /*interrupt call ends
```

❖ Interrupt Instructions work in 2 modes::

- ✓ User Mode      mode bit 1
- ✓ Kernel Mode      mode bit 0

❖ Types of interrupts::

➤ **S/W Interrupts**

Interrupts generated through software.

e.g Divide by zero.

Other Exceptions in your program.

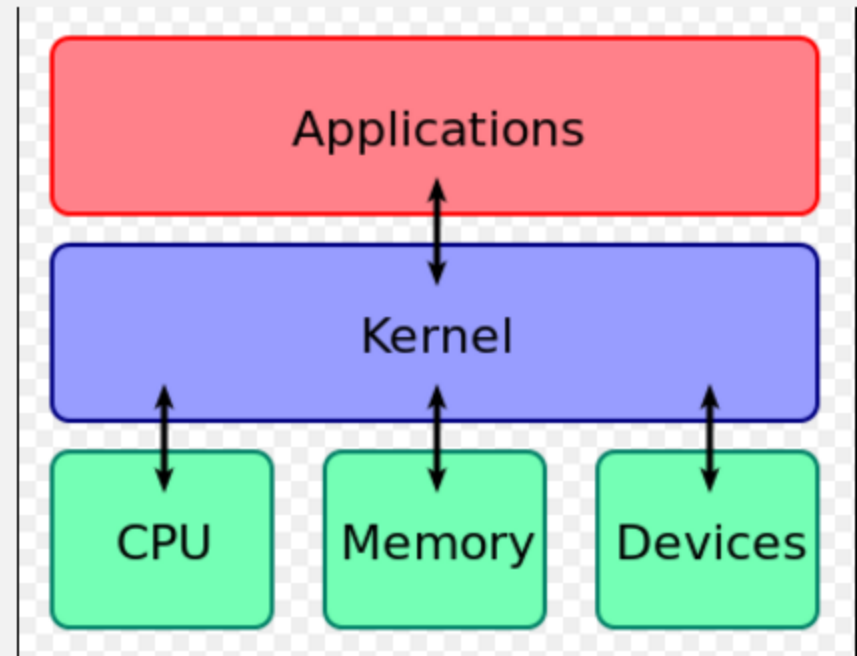
➤ **H/W Interrupts**

Interrupts generated through hardware.

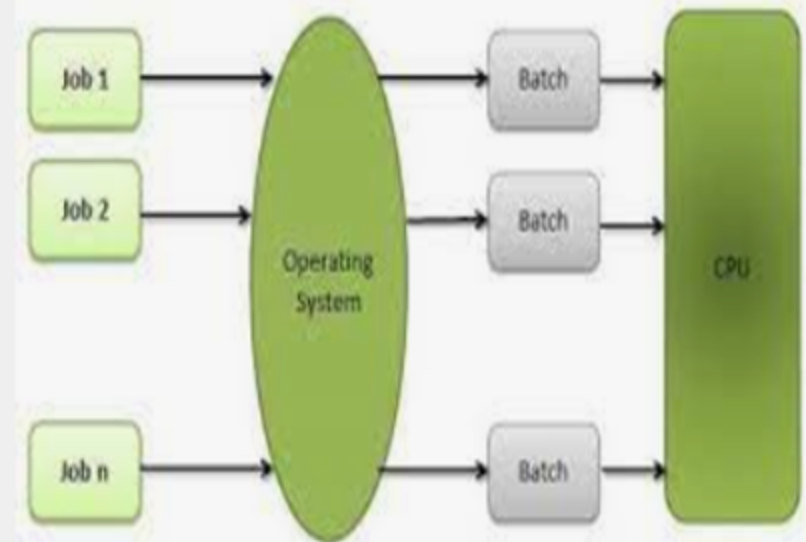
e.g Timer

# Steps To Load Kernel Into Memory

- When we switch on computer, control will go to ROM, on which *Bootstrap loader* is present.
- Job of bootstrap loader is to read 0<sup>th</sup> sector of hard disk, on which *boot loader* is present.
- Job of boot loader is to load *kernel* in to memory.



- ❖ No interaction between user and the computer.
- ❖ The user submits a job (written on cards or tape) to computer operator.
- ❖ Computer operator then places a batch of several jobs on an input device.
- ❖ Jobs are batched together by type of languages and requirement.
- ❖ Multiple users can share the batch systems.



## ❖ **Advantages::**

It is easy to manage large work repeatedly done.

## ❖ **Disadvantages::**

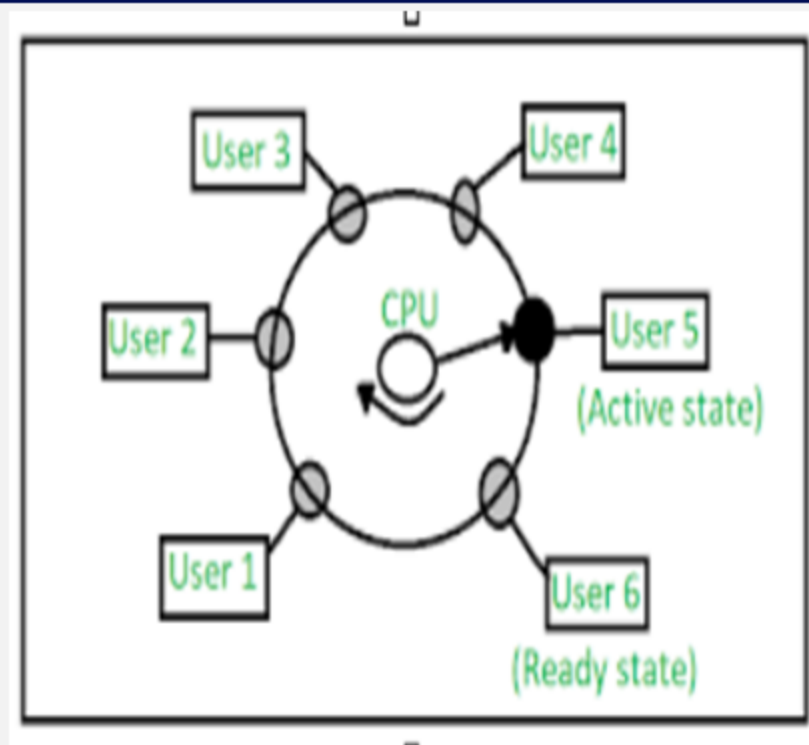
- The computer operators should be well known with batch systems.
- Batch systems are hard to debug.
- It is sometime costly.
- Some times the idle time in batch system is very high due to external machine speed .
- The other jobs will have to wait for an unknown time if any job in the batch fails.
- It is very difficult to guess or know the time required by any job to complete.

## Examples ::

Payroll System,  
Bank Statements etc.



- ❖ A time sharing system allows many users to share the computer resources simultaneously.
- ❖ The time sharing system provides the direct access to a large number of users where CPU time is divided among all the users on scheduled basis.
- ❖ The OS allocates a set of time to each user called as time slice/time quantam.
- ❖ When this time is expired, it passes control to the next user on the system. The time allowed is extremely small and the user gets the impression that they each have their own CPU and they are the sole owner of the CPU.

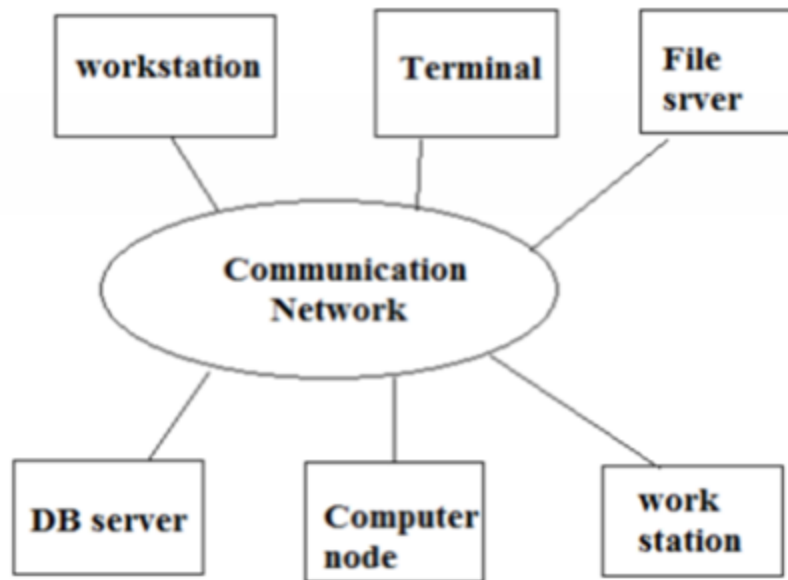


## Advantages ::

- Each task gets an equal opportunity.
- CPU idle time can be reduced.
- Multiple programs can be executed.

## Disadvantages ::

- One must take care of security and integrity of user programs and data.



## Advantage::

- With resource sharing facility user at one site may be able to use the resources available at another site.
- Speedy exchange of data with one another via electronic mail.
- If one site fails in a distributed system, the remaining sites can potentially continue operating.
- Better service to the customers.

## Disadvantage::

- Security problem due to sharing.
- Some messages can be lost in the network system.
- Bandwidth is another problem if there is large data then all network wires to be replaced

- ❖ In distributed OS data is stored and processed at multiple locations.
- ❖ Distributed systems comprises of many computers that are inter connected by communication networks.

- ❖ These systems are used when time requirements are very strict like missile systems, air traffic control systems, robots etc.
- ❖ Response time of these systems is very critical.

eg

--Railway reservation system, Satellite tracking system.

Session Ends.....